

LED Control

LED Control

- 1. LED Control Overview
- 2. Core Concepts
 - 2.1 Common Anode Configuration
 - 2.2 Core API
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 File Header and Macros
 - 5.2 `setup()` — Initialization
 - 5.3 `loop()` — Main Loop
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Behavior
 - 6.3 Verification Methods
- 7. Common Issues

1. LED Control Overview

GPIO is the most fundamental peripheral interface of MCUs. Pins can be configured as output mode via software to drive external circuits. This experiment demonstrates the basic usage of three core functions (`pinMode`, `digitalWrite`, `delay`) by blinking two onboard LEDs synchronously at 1Hz.

Typical Application Scenarios:

Application Type	Example
Digital Output	LED blinking, relay control, buzzer driving
Digital Input	Button detection, switch state reading, sensor digital signals
Bidirectional I/O	I2C bus (SDA), single-wire protocol
Interrupt Input	External event triggering
Analog Input	ADC sampling
Multiplexed Functions	UART / SPI / I2C / PWM and other peripheral signals

2. Core Concepts

2.1 Common Anode Configuration

The onboard LEDs use **common anode** configuration: anode is fixed at 3.3V, cathode connects to GPIO through a current-limiting resistor.

VDD(3.3V) → LED anode → LED cathode → current-limiting resistor → GPIO pin

GPIO Output	LED State	Principle
LOW (0V)	ON	GPIO pulls internal to GND, current flows through LED
HIGH (3.3V)	OFF	GPIO at same potential as VDD, no current

`digitalwrite(pin, LOW)` = ON, which is the opposite of the intuition "1=ON". Refer to the table above to understand.

2.2 Core API

`pinMode(pin, mode)` — Configures GPIO direction.

mode	Description
OUTPUT	Push-pull output, can drive ~10mA, suitable for LED
INPUT	High-impedance input (floating level is uncertain, not used alone)
INPUT_PULLUP	Input + internal ~45kΩ pull-up

`digitalwrite(pin, value)` — Writes GPIO output level. `LOW` = 0V, `HIGH` = 3.3V. Writes directly to register, single call < 0.1μs.

`delay(ms)` — Blocking delay, underlying is FreeRTOS `vTaskDelay()`: task enters Blocked state and yields CPU, automatically wakes up when expired.

Do not use `for` loop busy-wait — it will starve IDLE and trigger watchdog.

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
43	LED0	OUTPUT	Onboard LED, common anode, <code>LOW</code> =ON
44	LED1	OUTPUT	Same as above

Both LEDs are onboard, no external components required.

4. Project Architecture and Flow

```
setup()                // Executes only once
├─ pinMode(43, OUTPUT)  // GPIO43 push-pull output
└─ pinMode(44, OUTPUT)  // GPIO44 push-pull output

loop()                 // Infinite loop
├─ digitalWrite(43, LOW) // ON
├─ digitalWrite(44, LOW)
├─ delay(500)           // Hold for 500ms
├─ digitalWrite(43, HIGH) // OFF
├─ digitalWrite(44, HIGH)
└─ delay(500)           // Hold for 500ms → back to start
```

5. Source Code Details

Full source code: [Source Code](#) → [Arduino](#) → [1.Basic Peripherals](#) → [1.GPIO_LED](#) → [1.GPIO_LED.ino](#)

5.1 File Header and Macros

Define pin numbers first, then reference all GPIO operations through macro names for better readability and easier modification.

```
#define LED0_PIN  43  // Onboard LED0
#define LED1_PIN  44  // Onboard LED1
```

`digitalwrite(LED0_PIN, LOW)` is instantly recognizable as operating LED0, compared to `digitalwrite(43, 0)`; if hardware revision moves LED to GPIO48, only one macro line needs changing.

5.2 `setup()` — Initialization

Configure both GPIOs as output mode so they can drive the LEDs.

```
void setup() {
  pinMode(LED0_PIN, OUTPUT);
  pinMode(LED1_PIN, OUTPUT);
}
```

`pinMode(pin, OUTPUT)` calls `gpio_set_direction()` underneath, setting GPIO to push-pull output with ~10mA drive current.

5.3 `loop()` — Main Loop

ON then OFF, each delay 500ms. The repeated execution of `loop()` creates 1Hz blinking. In common anode configuration, `LOW`=ON — when GPIO pulls low, current flows through LED to GND, LED illuminates.

```
void loop() {  
    // ON - LOW: GPIO pulls to GND, LED has voltage difference  
    digitalWrite(LED0_PIN, LOW);  
    digitalWrite(LED1_PIN, LOW);  
    delay(500);  
  
    // OFF - HIGH: GPIO pulls to 3.3V, LED at equal potential  
    digitalWrite(LED0_PIN, HIGH);  
    digitalWrite(LED1_PIN, HIGH);  
    delay(500);  
}
```

`digitalWrite` writes directly to GPIO output register, single call $< 0.1\mu\text{s}$. `delay(500)` uses `vTaskDelay()` underneath, task enters Blocked state and yields CPU, automatically wakes up after 500ms, completely different from busy-wait.

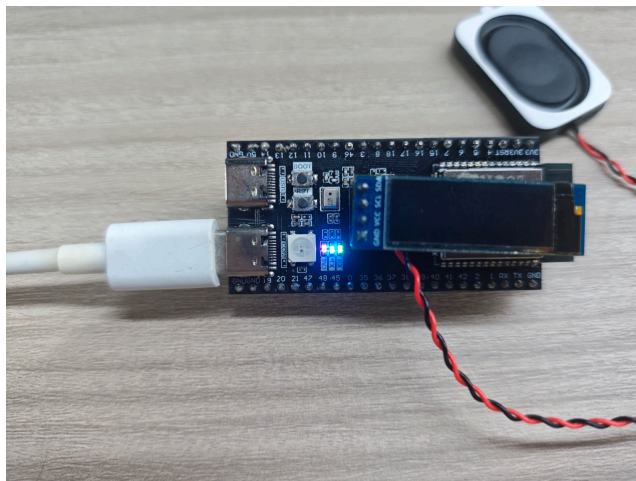
6. Operation Flow

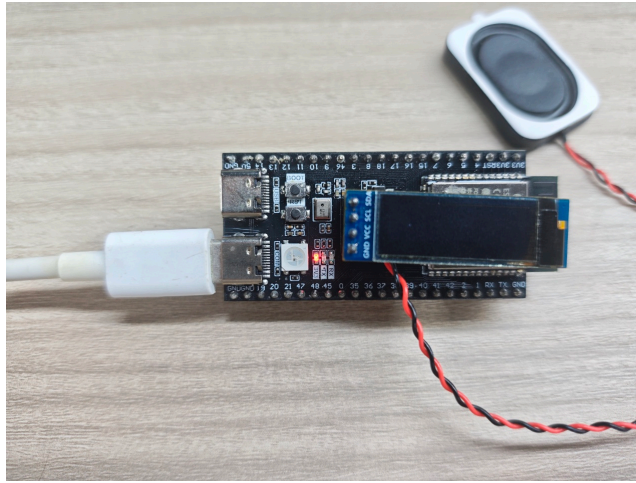
6.1 Build and Flash

Arduino flash process: See `Arduino` → 1. Before Development → 2. Build and Flash.

6.2 Normal Behavior

After power-on, both LEDs blink synchronously at 1Hz: ON 500ms → OFF 500ms → repeat.





6.3 Verification Methods

Operation	Expected Result
Power on and observe	Both LEDs blink synchronously, period ~1s
Change <code>delay(500)</code> to <code>delay(100)</code>	Blinking significantly faster, ~5Hz
Comment out only one <code>digitalWrite</code>	Only one LED blinks

7. Common Issues

Phenomenon	Cause	Solution
Build fails "LED0_PIN was not declared"	Macro definition commented or deleted	Confirm <code>#define LED0_PIN 43</code> exists
LED does not light up	Pin number inconsistent with board silkscreen	Confirm GPIO43/44
LED always stays on	Common anode/common cathode reversed	Swap <code>LOW</code> / <code>HIGH</code>
Button unresponsive during <code>delay()</code>	<code>delay</code> is blocking	See button detection experiment for non-blocking solutions